

Vergleich verschiedener Dateisysteme für den SD-Karten-Einsatz in eingebetteten Systemen

Michael Kolorz (miko8278@bht-berlin.de)

4. November 2026

Erstprüfer: Prof. Dr. Peter Gober Zweitprüfer: Prof. Dr. David Dietrich

Themen des Vortrags

1. Überblick und Motivation
2. Hardwaredesign
3. Software
   - 3.1 SPI-SD-Treiber
   - 3.2 FatFs und littlefs
4. Dateisystemtests
   - 4.1 Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge
   - 4.2 Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge und manipulierter Schreibfunktion
   - 4.3 Analyse von pseudo-randomisierter Zeichenfolge bei Anhang an Datei
   - 4.4 Verhalten der Dateisysteme bei Stromunterbrechungen
5. Fazit

Überblick

- Design einer Platine, die einen STM32-G431 über eine SPI-Schnittstelle mit einem SD-Kartenslot verbindet
- Softwareseitige Implementation eines SPI-SD-Kartentreibers für blockweise Lese- und Schreibfunktionen
- Nutzung dieser Funktionen für eine Anbindung von zwei Dateisystemen: FatFs (FAT32) und littlefs
- Vergleich dieser beiden Dateisysteme über eigens erstellte Tests

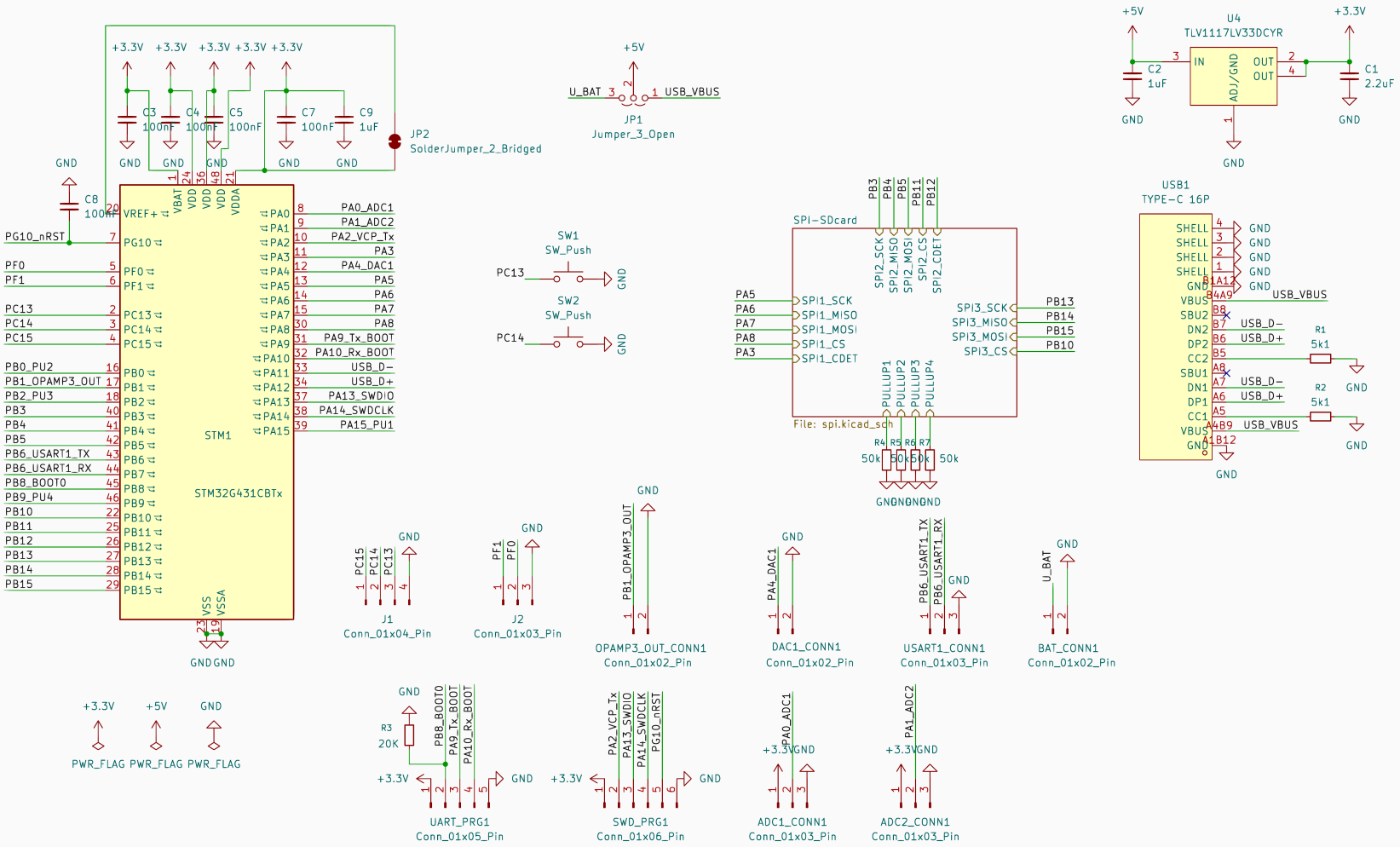
Motivation technisch

- Wie größere Mengen an bspw. Sensordaten speichern oder Daten ausgeben?
- Einplatinencomputer RPI
  - Hoher Stromverbrauch (>200 mA bei 5 V vs 20 mA bei 5 V)
  - Betriebssystemoverhead / Echtzeitfähigkeit?
- Warum SD-Karten?
  - NOR-Flash 16 MB (Icsc) -> 1,9 $, 10,63 Eurocent pro MB
  - SD-Karte 32 GB (reichelt) -> 7 €, 0,0219 Eurocent pro MB

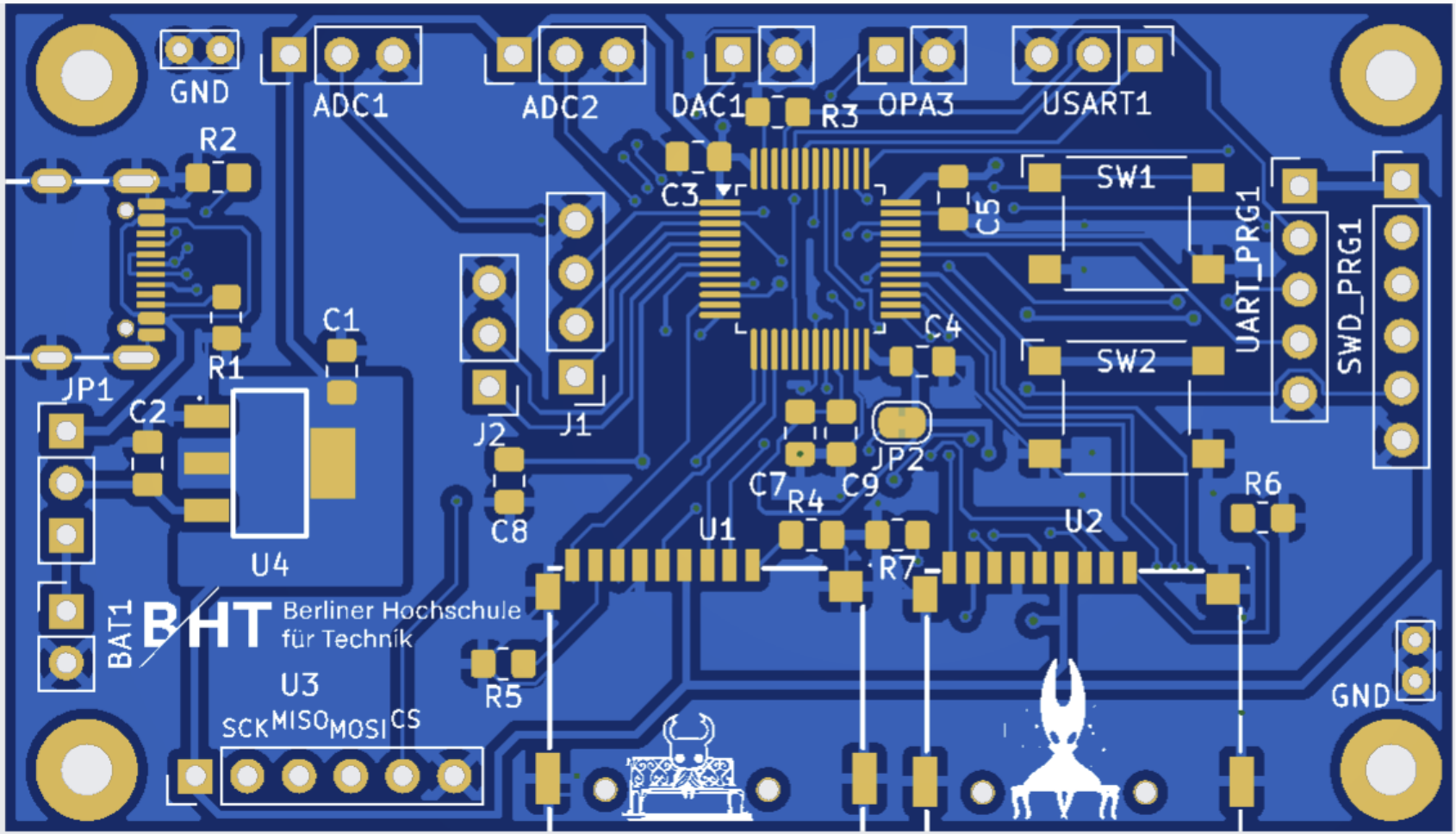
Motivation persönlich

- Wenig Interesse am Praktikumsthema Drehgeber
  - 3 Monate Erfahrung mit Embedded C++
- Interesse an PCB-Design
- Dateisysteme und Speicherung von Daten wenig Thema im Studium

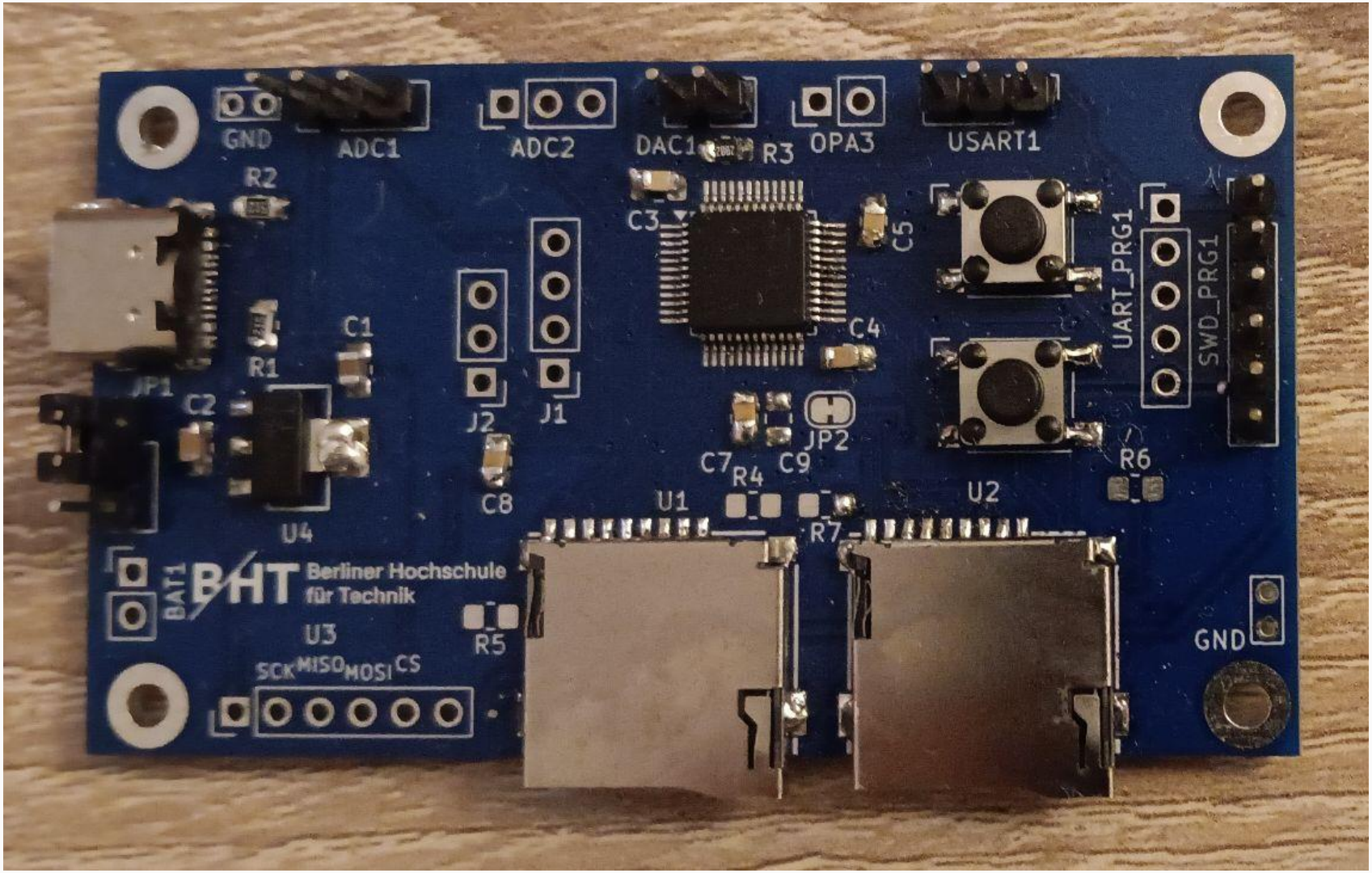
Hardware: Schaltplan



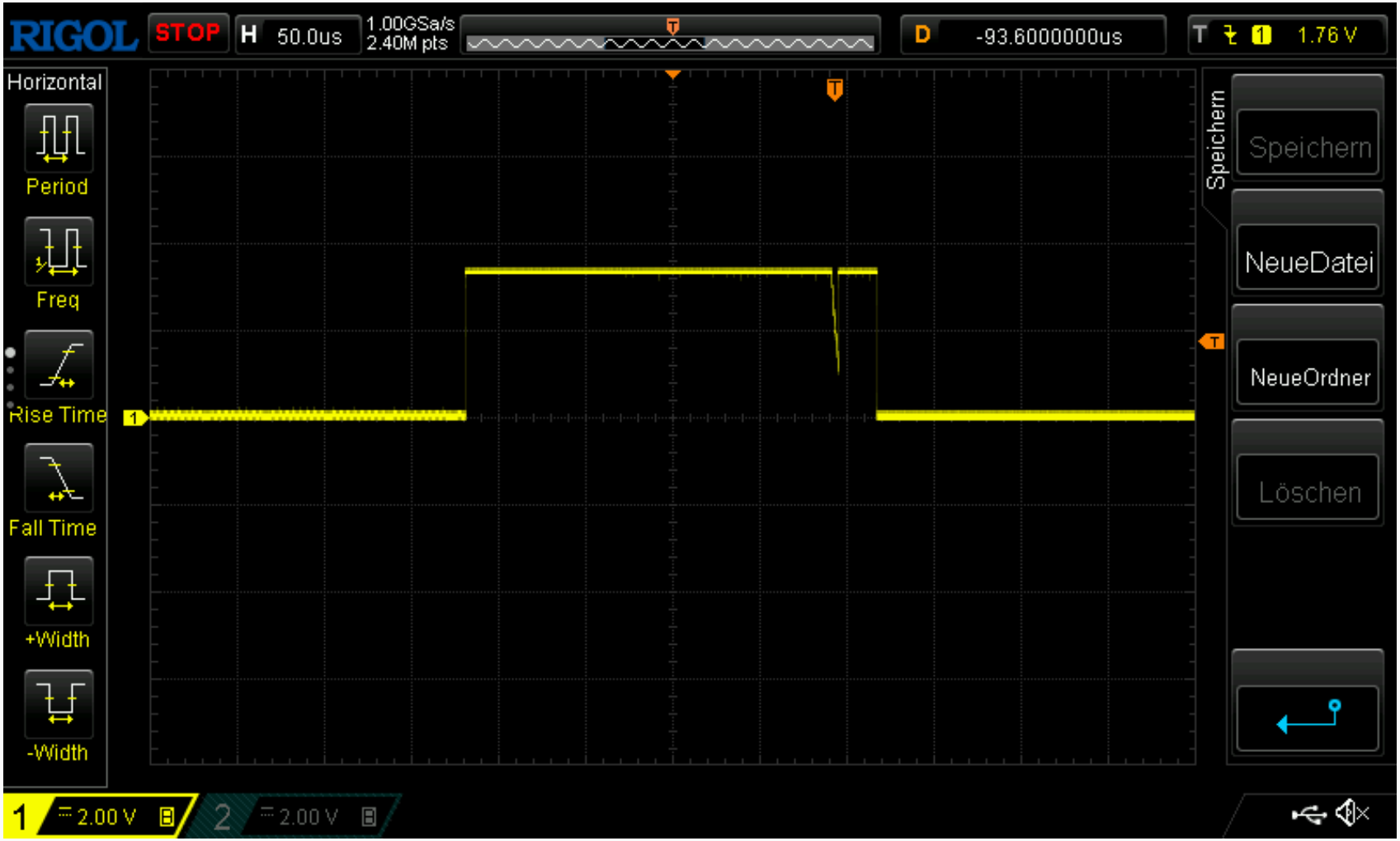
Hardware: Platine unbestückt



Hardware: Platine bestückt



Hardware: Problem 2. SD-Kartenslot MISO



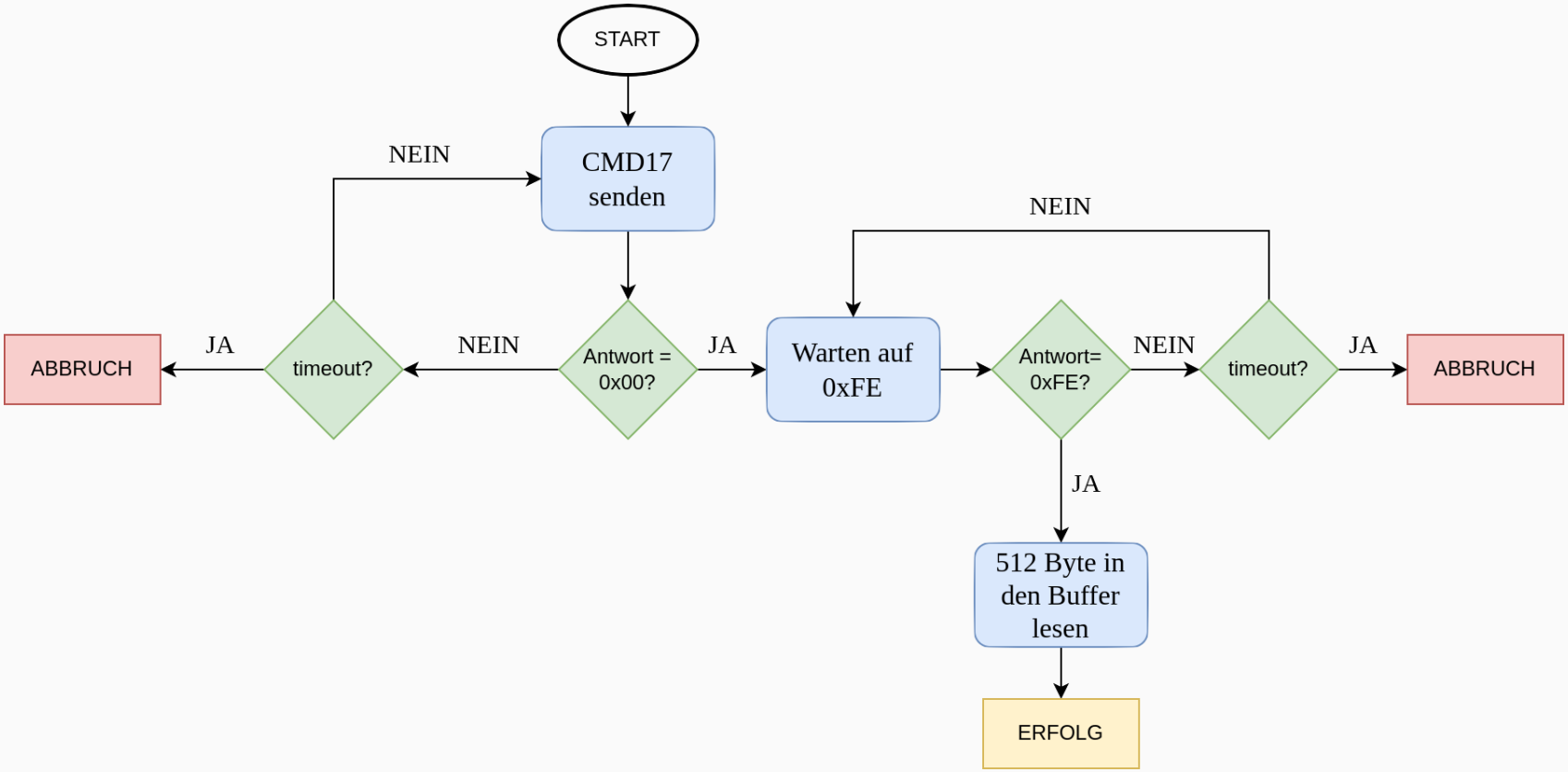
Software: VSCodium + CMake + Cortex-Debug

Demo

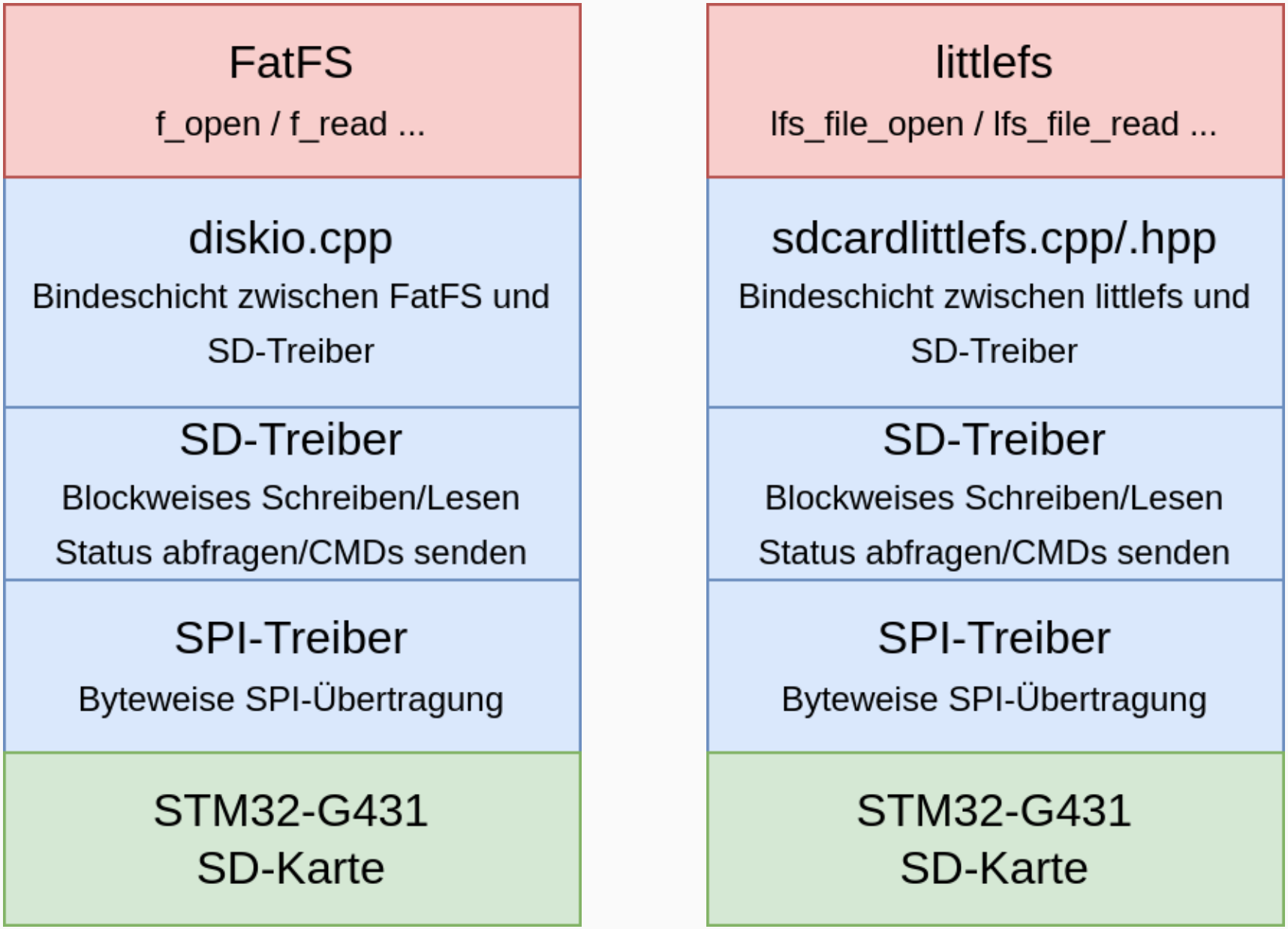
Software: SPI-SD-Treiber

- SPI konfiguriert auf byteweisen Transfer mit variabler SCLK
- SD-Treiber implementiert
  - Blockweises Lesen Schreiben (CMD17, CMD24)
  - Schreiben und Lesen von multiplen Blöcken (CMD18, CMD25)
  - Abfrage der CSD (Card-Specific Data, CMD9) -> hauptsächlich SD-Kartengröße

Software: SPI-SD-Treiber blockweises Lesen (CMD17)



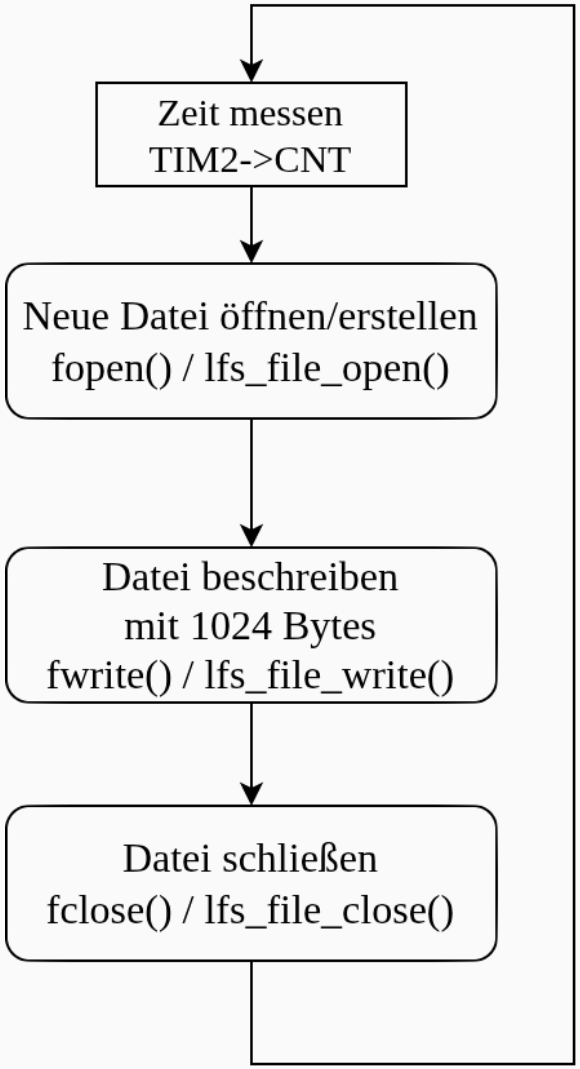
Dateisystem: FatFs und littlefs Implementierungsstack



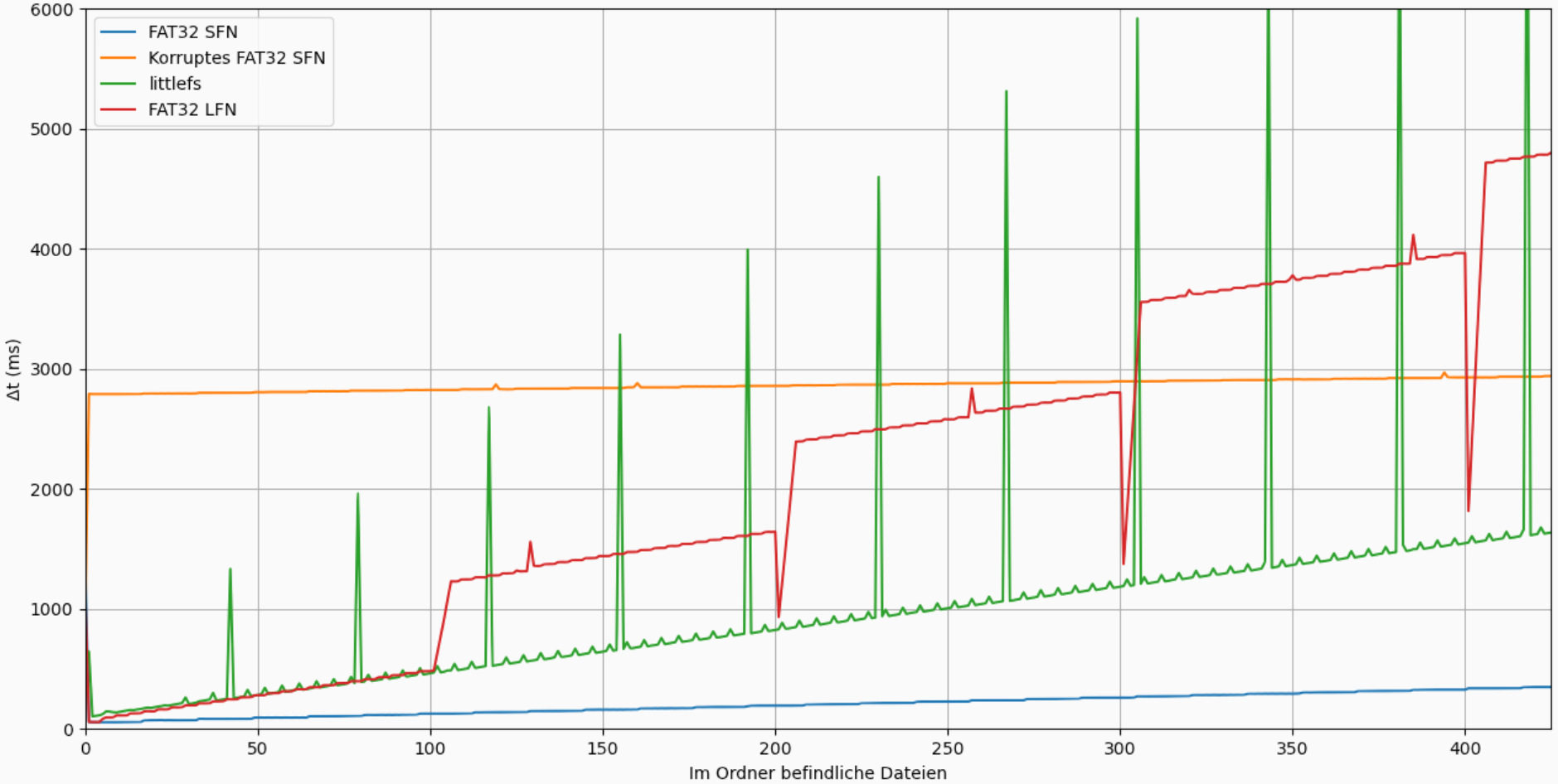
Dateisysteme: FatFs und littlefs

- FatFs:
  - C-Bibliothek von Chan für alle FAT-Dateisysteme auf Mikrocontrollern
  - Out-of-Box Kompatibilität mit allen Betriebssystemen
  - Kein Journaling: Stromausfälle oder anderweitige Schreibfehler können das Dateisystem beschädigen
- littlefs:
  - Ebensfalls in C geschriebenes Dateisystem, das speziell für Flash-Speicher entwickelt und optimiert wurde
  - Dynamisches Wear-Leveling verteilt Schreibzugriffe
  - COW und two-block log für Dateisystemintegrität
  - Von PC-Betriebssystemen nicht nativ unterstützt (FUSE-Wrapper vorhanden)

[Versuch1] Dateisystemtests: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge



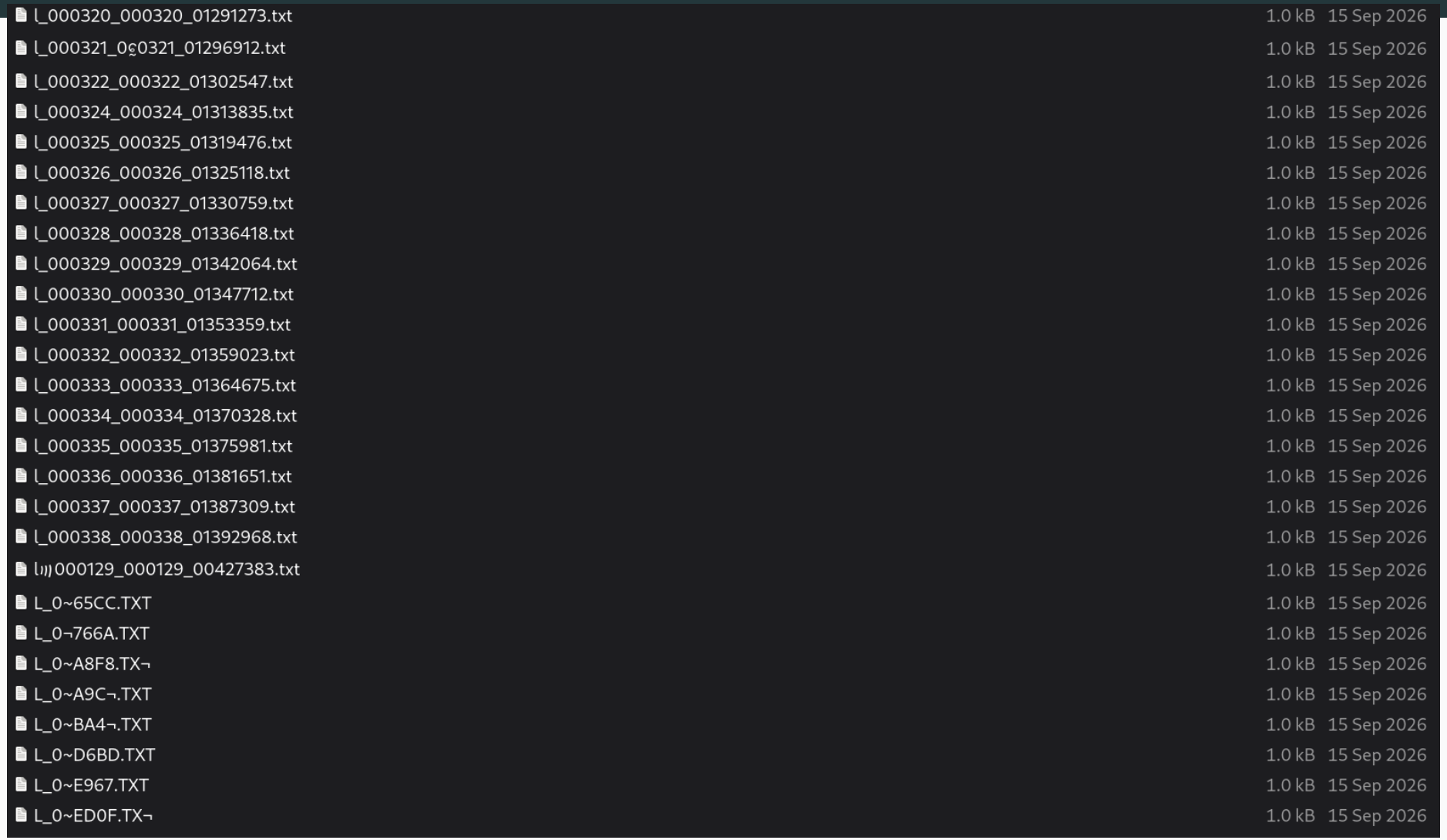
[Versuch1] Dateisystemtests: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge



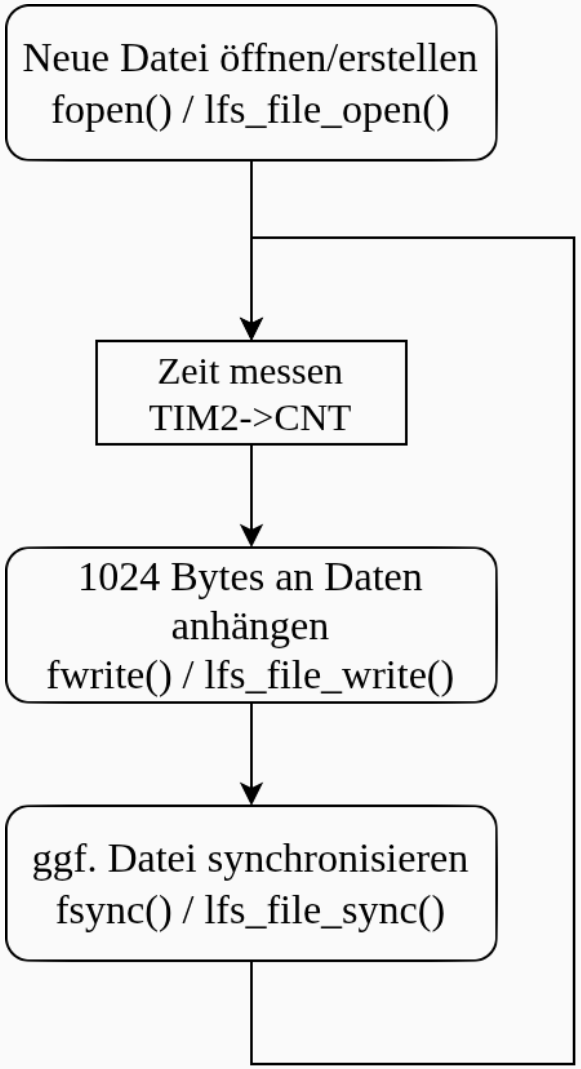
[Versuch2] Dateisystemtests: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge und manipulierter Schreibfunktion

```
// send 512 byte blockdata
for (uint16_t i = 0; i < 512; i++) {
    //Injecting errors every 5000 byte
    if(err_cnt == 5000)
    {
        //some random data
         SpiDriver<Config::SpiBase>::Transfer(0xAA);
         err_cnt = 0;
         continue;
     }
    err_cnt++;
    SpiDriver<Config::SpiBase>::Transfer(buffer[i]);
}
```

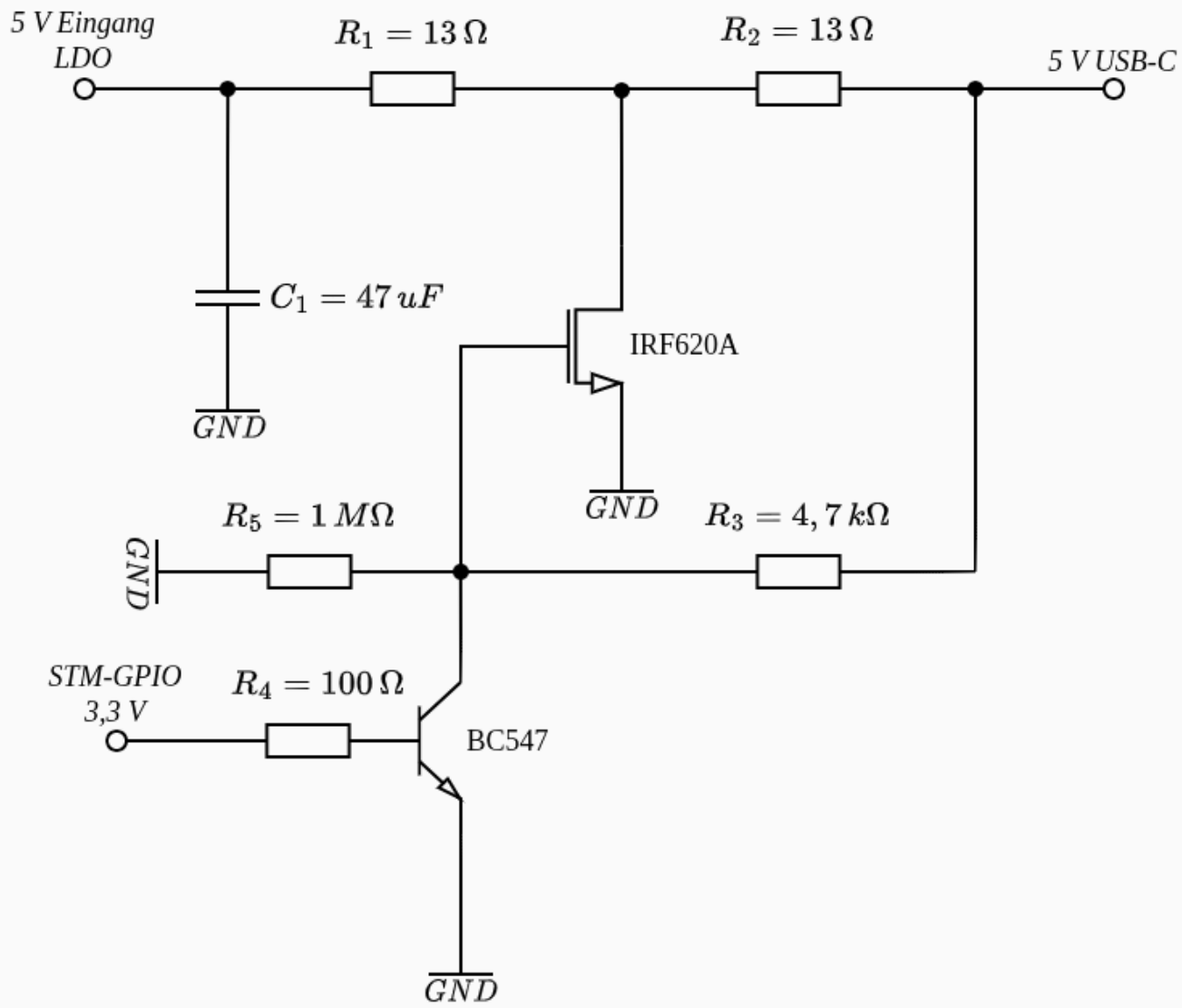
[Versuch2] Dateisystemtests: Analyse der Schreibfehlerraten mittels bekannter Zeichenfolge und manipulierter Schreibfunktion



[Versuch3] Dateisystemtests: Analyse von pseudo-randomisierter Zeichenfolge bei Anhang an Datei



[Versuch4] Dateisystemtests: Verhalten der Dateisysteme bei Stromunterbrechungen



Fazit

- Hardware grundsätzlich funktional -> Hauptproblem Spannungsversorgung SD-Karte
- Durch eigenen SD-Kartentreiber hohes Maß an Kontrolle möglich: Schlanke Funktionalität -> minimaler RAM/Flashverbrauch.
- FatFs überzeugt auf der SD-Karte durch einfache Handhabbarkeit, Anbindung und minimalen Overhead
- Littlefs bringt integrierten Fehlerschutz mit, führt aber zu Laufzeitkosten, die eine Echtzeitfähigkeit des Systems einschränken
    - Sinnvoller auf echter Flashhardware statt SD-Karte

Danke für die Aufmerksamkeit

Fragen & Antworten